

媒体与认知课程上机作业中期报告：基于SigLIP的小规模图文检索系统

徐一 2023010514 无32

1 项目背景

多模态学习（Multimodal Learning）是当前人工智能领域的研究热点，其中视觉与语言的融合任务，如图文匹配、图文检索、图文生成等，具有广泛的应用前景。

CLIP（Contrastive Language-Image Pretraining）是 OpenAI 提出的多模态预训练方法，利用图像-文本对比学习，使模型能自动将图像和文本嵌入到同一语义空间中，进而支持多种下游任务。然而该方案存在诸多问题，如强依赖大 batch size。

SigLIP（Sigmoid Loss for Language-Image Pretraining）是由 Google Research 提出的一种视觉-语言预训练方法，是对经典对比学习框架（如 CLIP）的重要改进。

2 任务目标

1. 理解 CLIP 与 SigLIP 在损失函数上的差异。
2. 使用Pytorch实现图像编码器、文本编码器和 SigLIP 损失函数。
3. 在 Flickr8k 数据集上训练模型，并报告 Text-to-Image 与 Image-to-Text 的 Recall@1/5/10。
4. 分析模型在多模态语义对齐方面的效果及可视化结果。
5. 基于 google/siglip-base-patch16-224 预训练模型，体验图文跨模态对比学习的核心结果——图文交叉相似度矩阵。

3 技术路线与方法

3.1 数据及选择

使用 Flickr8k 数据集，数据格式为图像 + 文字描述（1-5 条）。

3.2 模型结构

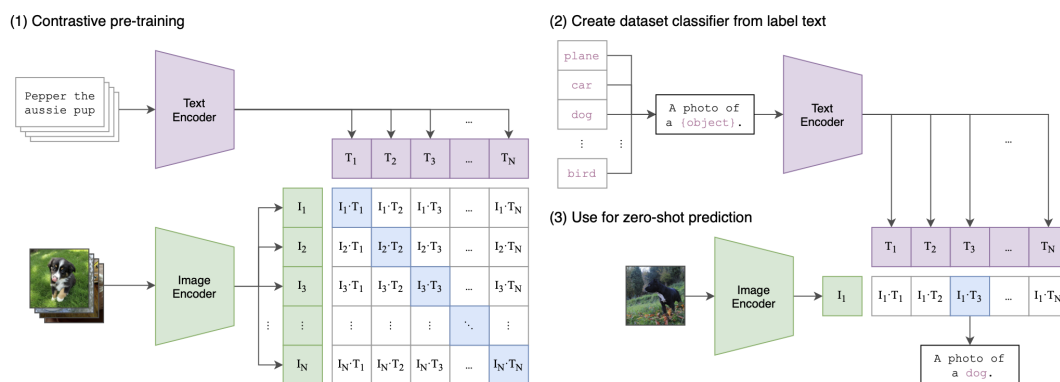


图 1: 模型结构

● 图像编码器【Task1】

- 使用 ResNet18 进行特征提取，输出特征向量。
- 添加线性层映射到共享语义空间。

● 文本编码器【Task2】

- 使用 MLP 或 Transformer 提取文本表示。
- 线性层将文本特征映射到相同维度的共享空间。

• SigLIP损失函数【Task3】

- SigLIP Loss (Sigmoid-based)

设图像编码为 z_i^I ，文本编码为 z_j^T ，相似度仍采用余弦相似度。

设一个 batch 中包含 $|\mathcal{B}|$ 个图文样本对，图像编码为 x_i ，文本编码为 y_j ，二者的相似度定义为：

$$z_{ij} = \frac{x_i^\top y_j}{\|x_i\| \|y_j\|}$$

其中， t 为可学习的温度系数， b 为可学习的偏置项。定义图文匹配标签为：

$$s_{ij} = \begin{cases} +1, & i = j \\ -1, & i \neq j \end{cases}$$

则对于任意一对 (i, j) ，SigLIP 的成对损失为：

$$\mathcal{L}_{ij} = \log(1 + \exp(s_{ij}(-tz_{ij} + b)))$$

因此，整个 batch 上的 SigLIP 损失为：

$$\mathcal{L}_{\text{SigLIP}} = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{j=1}^{|\mathcal{B}|} \mathcal{L}_{ij}$$

即

$$\mathcal{L}_{\text{SigLIP}} = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{j=1}^{|\mathcal{B}|} \log(1 + \exp(s_{ij}(-tz_{ij} + b)))$$

4 训练流程

1. 图像 + 文本输入模型
2. 提取图像特征 / 文本特征
3. 归一化后计算余弦相似度矩阵
4. 计算损失并反向传播
5. 每轮评估 Top-K Accuracy / Recall@K 检索准确率，保存最优模型

5 实验环境

1. GPU型号：NVIDIA GeForce RTX 5090
2. PyTorch版本：2.11.0

6 结果分析与可视化

6.1 Sanity Check

运行命令

```
1 $ python train_siglip.py --dry-run --epochs 2 --batch-size 32 --num-workers 0
```

发现训练可以正常运行，运行结果保存至outputs/siglip_resnet_transformer/目录下。

6.2 训练 Resnet + Transformer

运行命令

```
1 $ python train_siglip .py \
2 $ --data-dir Flickr8k \
3 $ --text-encoder transformer \
4 $ --epochs 20 \
5 $ --batch-size 64 \
6 $ --output-dir outputs/resnet.transformer
```

lr 为默认值 1×10^{-4} 。训练loss曲线和最终测试集输出结果如下，运行结果保存至`outputs/resnet_transformer/`目录下。

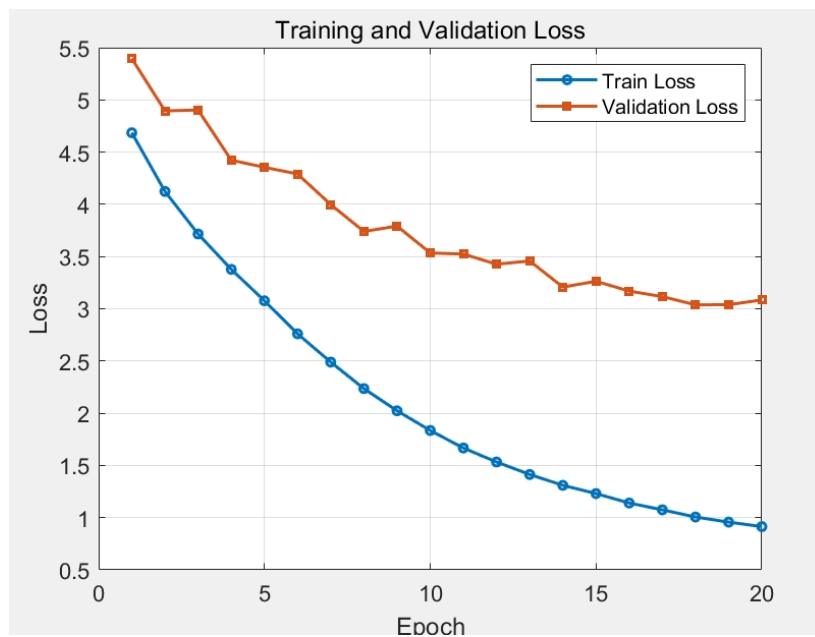


图 2: ResNet + Transformer 模型训练loss曲线

$final \ test_loss = 3.0497$ 。训练效果较好。

Model	Text-to-Image			Image-to-Text		
	R@1	R@5	R@10	R@1	R@5	R@10
ResNet + Transformer	11.84	30.65	42.88	12.04	31.61	42.83

表 1: ResNet + Transformer 模型在 Flickr8k 上的检索结果

6.3 轻量文本基线

运行命令

```
1 $ python train_siglip .py \
2 $ --data-dir Flickr8k \
3 $ --text-encoder mlp \
4 $ --epochs 20 \
5 $ --batch-size 64 \
6 $ --output-dir outputs/resnet.mlp
```

lr为默认值 1×10^{-4} 。训练loss曲线和最终测试集输出结果如下，运行结果保存至outputs/resnet_mlp目录下。

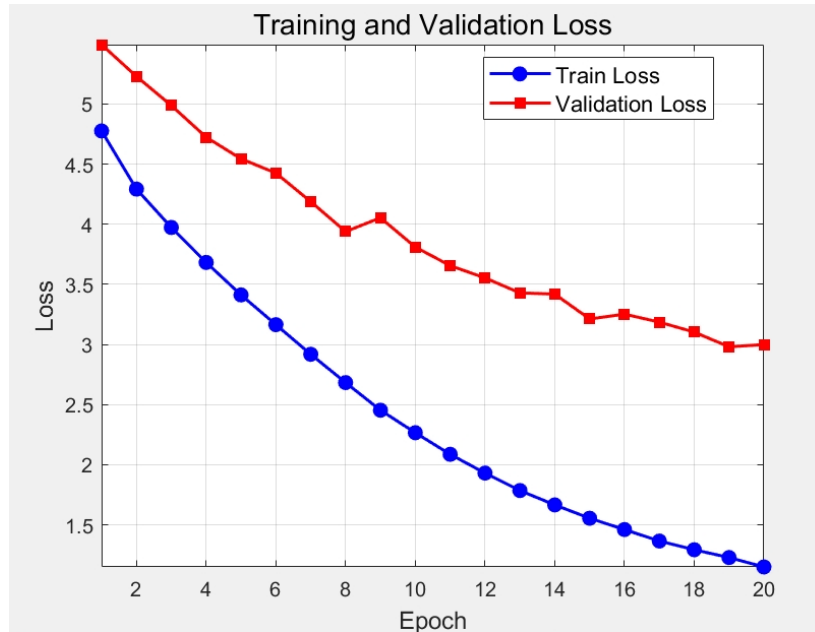


图 3: ResNet + MLP 模型训练loss曲线

final test_loss = 3.0192。训练效果较好。

Model	Text-to-Image			Image-to-Text		
	R@1	R@5	R@10	R@1	R@5	R@10
ResNet + MLP	11.15	31.07	42.91	11.37	30.91	42.74

表 2: ResNet + MLP 模型在 Flickr8k 上的检索结果

可见，两者性能接近，在 Text-to-Image 检索上轻量文本基线甚至略优于 ResNet + Transformer 模型。

分析原因可能为：

1. 图像编码器 ResNet18 参数过少、性能较差，影响了模型的性能上限；
2. 文本编码器采用平均池化，将所有 token 同等对待，使得模型退化，没有通过注意力机制学习到的顺序和交互信息；

3. Flickr8k 数据规模小，Transformer 学习不充分；

4. 学习率、温度、batch size等参数未进行充分调优，可能影响了模型的性能表现。

5. 预训练初始化的权重可能不适合当前任务，导致模型在小规模数据集上表现不佳。

笔者认为可能的优化方向有：

1. 微调训练超参数，增大epoch数；
2. 增强图像编码器，如增大 -image-width；
3. Transformer 引入其他更合适的池化方式。
4. 优化与训练初始化的策略。

6.4 可视化

编写文本 → 图像 Top-K 检索示例的可视化代码，见 visualize_topk.py。

运行命令

```
1 $ python visualize_topk.py \
2 $ --data-dir Flickr8k \
```

```

3 $ --checkpoint outputs/resnet_transformer/ best_siglip .pt \
4 $ --split test \
5 $ --topk 5 \
6 $ --num-queries 8 \
7 $ --output-dir outputs/topk_vis_transformer

```

得到 Top-5 检索结果，取几次检索结果如下图所示，图像保存在 /outputs/topk_vis_transformer/ 下。

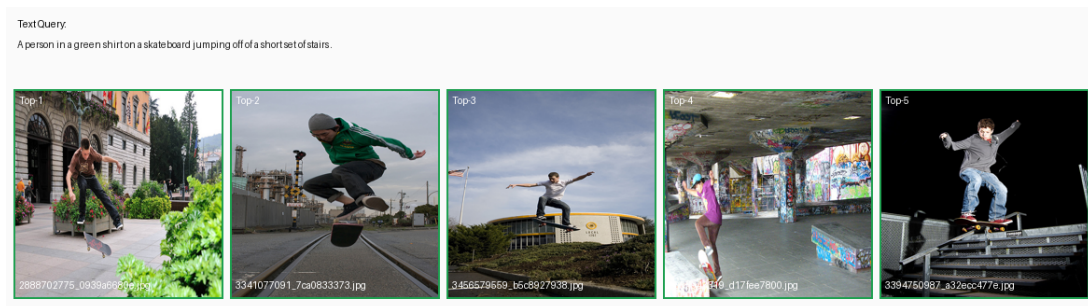


图 4: 文本 → 图像 Top-5 检索结果示例1

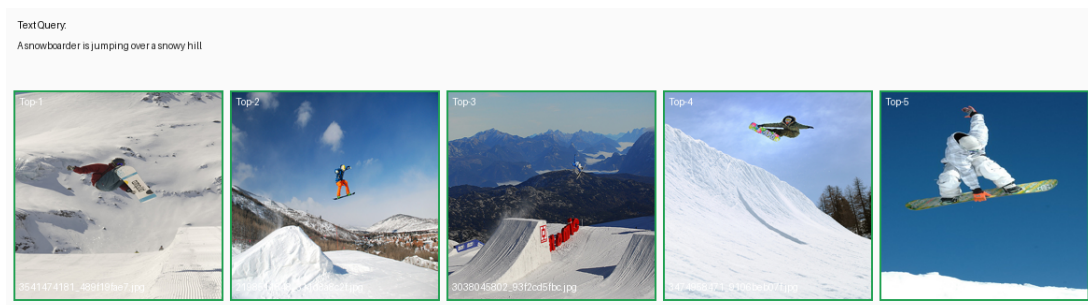


图 5: 文本 → 图像 Top-5 检索结果示例2

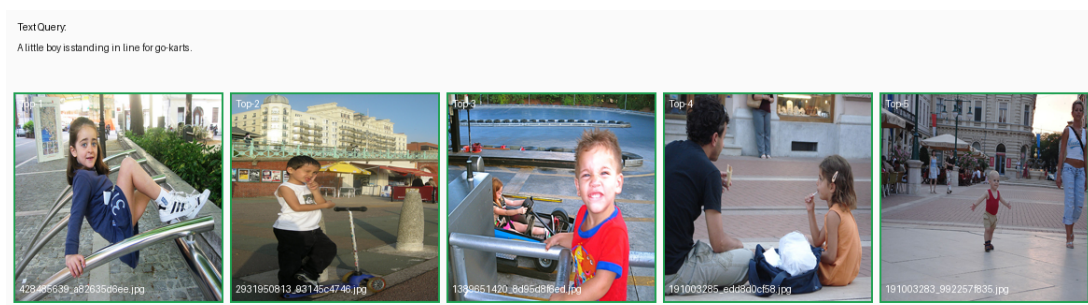


图 6: 文本 → 图像 Top-5 检索结果示例3

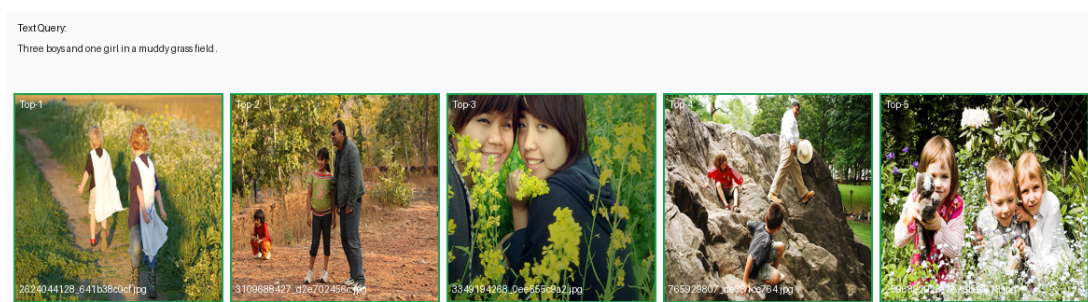


图 7: 文本 → 图像 Top-5 检索结果示例4

对比以上四次检索结果，可以发现前两个示例的检索较准确，而后两个较差。具体来说，基于当前的训练情况，模型在一些简单的场景（如单人、物体较少）上表现较好，但在复杂场景（如多人、多物体）尤其是对男女性别判断上存在较大问题。

7 图文交叉相似度矩阵

在visualization/目录下运行命令

```
1 $ pip install -r requirements.txt
2 $ export HF_ENDPOINT=https://hf-mirror.com
3 $ python app.py
```

访问 <http://localhost:7860>，取图文对数量为8，随机采样并生成热力图如下所示：



图 8: 图文交叉相似度矩阵可视化示例

数据分析：



图 9: 图文交叉相似度矩阵数据分析